Project Report: University ERP System

Course: CSE 201 – Advanced Programming

Submitted By: Achintya Shukla & Manan Saini

---

1. Project Overview

1.1 Introduction

The University ERP System is a desktop application created to manage the day-to-day activities of a university. I built it using Java Swing for the design and MySQL for saving data. The system works like a real university portal where three types of users—Students, Instructors, and Admins—can log in and do their tasks, like registering for courses or entering grades.

The main goal of this project was to learn how to build a "3-tier" application (connecting the screen, the code, and the database) while keeping the data safe. I also focused on solving problems like secure logins, stopping students from registering for full classes, and adding a "Maintenance Mode" to lock the system.

1.2 Technology Stack

- Language: Java (JDK 11 or higher)
- Framework (GUI): Java Swing (used FlatLaf to make it look modern)
- Database: MySQL 8.0
- Security: JBCrypt (to hide passwords)
- Build Tool: Maven

---

2. System Architecture

I designed the application in separate "modules" so the code remains clean. This means the code for the buttons and windows is kept separate from the code that talks to the database.

2.1 Database Design (Two Databases)

For better security, I decided to use two separate databases: one just for login info (auth_db) and one for the actual college data (erp_db).

A. Authentication Database (auth_db)

This database only handles user logins. It keeps the passwords separate from the student records so they are safer.

Table Name	Description	Key Columns
users_auth	The main table for storing users.	user_id (PK), username, password_hash, role

B. ERP Database (erp_db)

This database holds all the actual information about the university.

Table Name	Description	Key Columns
courses	List of all available subjects.	course_id (PK), code (e.g., CSE201), title, credits
sections	The specific classes scheduled for a course.	section_id, course_id (FK), instructor_id (FK), room, capacity
students	Details about the student.	user_id (FK), roll_no, program, year
instructors	Details about the teacher.	user_id (FK), department, title
enrollments	Links a student to a section (Many-to-Many).	enrollment_id, student_id (FK), section_id (FK)
grades	Stores the marks for exams.	enrollment_id (FK), component, score
settings	Saves system options (like Maintenance Mode).	setting_key (PK), setting_value

2.2 Application Structure

I organized my Java files into different packages:

- **edu.univ.erp.ui: Contains the screens (like LoginWindow and StudentPanel).**
- **edu.univ.erp.service: Contains the logic (like checking if a class is full).**
- **edu.univ.erp.auth: Handles password checking.**
- **edu.univ.erp.data: Manages the connection to MySQL.**

---

3. Role-Based Features

The app looks different depending on who logs in.

3.1 Admin Module

The Admin is the main controller of the system.

- **Manage Users: Admins can add new Students and Instructors. The code automatically creates a secure login for them and saves their profile.**
- **Course Scheduling: Admins create "Sections" for courses, assign a teacher, and set the room capacity.**
- **System Control: Only Admins can turn on "Maintenance Mode."**

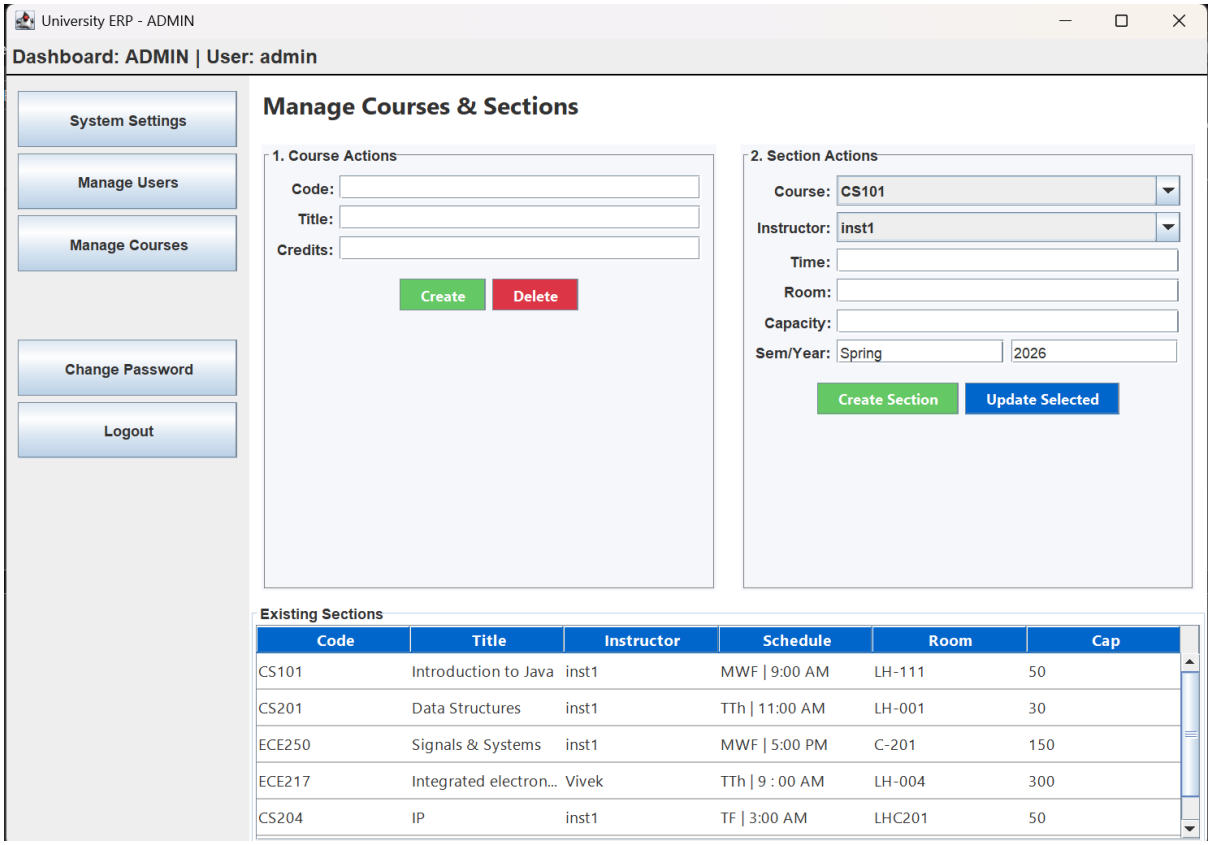

Figure 1: Admin Dashboard for managing system users.

3.2 Student Module

Students use the app mainly to pick their classes.

- **Course Catalog: They can see a list of all open courses.**
- **Registration: They can click to register, but the system checks if the class is full or if they already took it.**
- **Transcripts: They can see their grades once the teacher uploads them.**

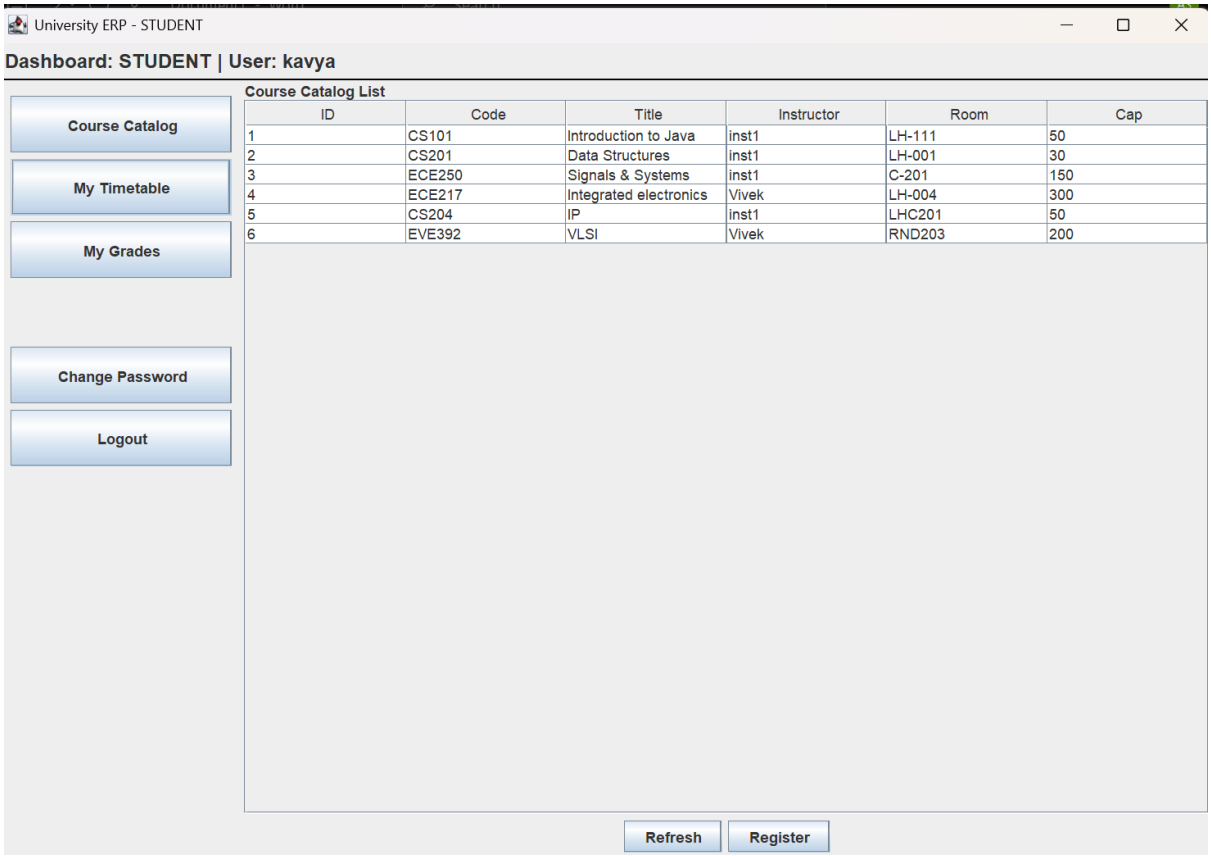

Figure 2: Student Registration Portal showing course list.

3.3 Instructor Module

Instructors use the app to manage their classes.

- **Class Roster: They can see a list of every student in their class.**
- **Grading: They can enter scores for Quizzes and Exams. The grades update instantly for the student**

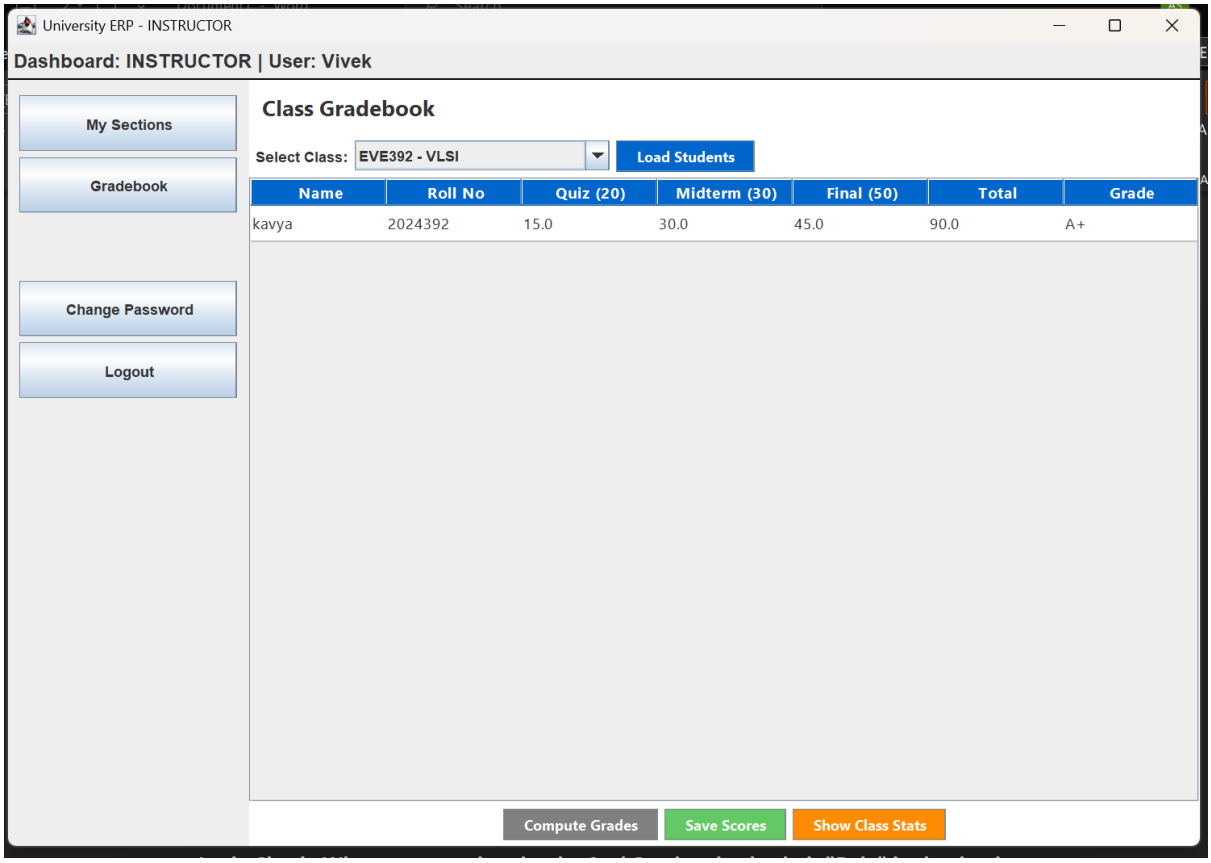

Figure 3: Instructor Interface for entering student grades.

---

4. How the Rules Work

This section explains the logic I wrote to make sure the system follows the rules.

4.1 Security (Roles)

I made sure users can't see screens they aren't supposed to.

- **Login Check: When someone logs in, the AuthService checks their "Role" in the database.**
- **Routing: The code uses a simple switch-statement:**
  - **If ROLE == 'ADMIN', it opens the Admin Panel.**
  - **If ROLE == 'STUDENT', it opens the Student Panel.**
- **Isolation: Since the panels are different Java files, a Student can't just "click back" to get to the Admin screen.**

4.2 Maintenance Mode

I added a feature called "Maintenance Mode" so Admins can stop changes while they work on the system.

- **How it works: There is a setting in the database called maintenance_on.**
- **The Check: Before a student registers or drops a course, my code checks this setting:**

Java

```
if (adminService.isMaintenanceMode()) {
    throw new Exception("System is in Maintenance Mode. Actions disabled.");
}
```

If it's ON, the app stops them and shows an error message

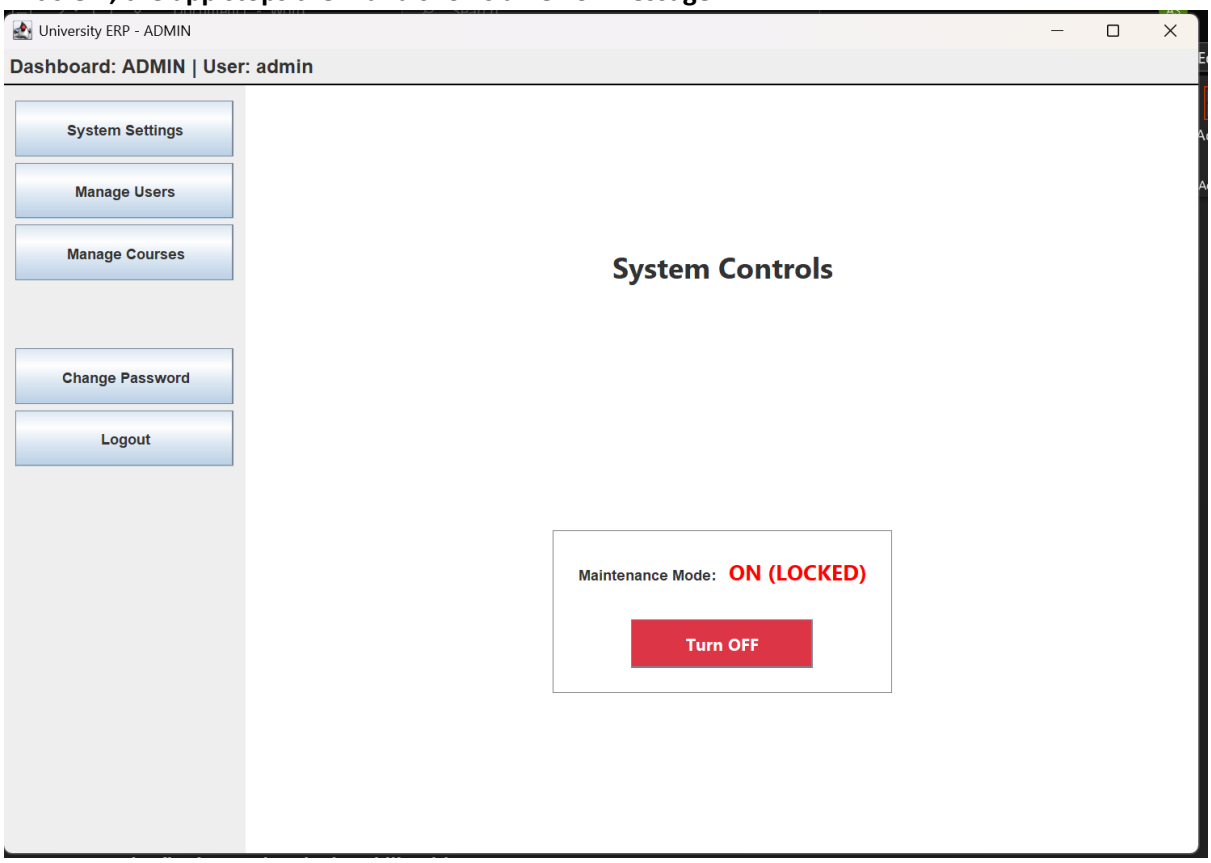

Figure 4: Admin Settings Panel where Maintenance Mode is toggled.

4.3 Grading Logic

For grading, I didn't just add up the numbers. I used a weighted average system.

- The Rule:
    - Quiz: 20%
    - Midterm: 30%
    - Final: 50%
- The Code:

The InstructorService pulls the scores from the database. If a grade is missing, it counts as zero. The final score is calculated like this:

Total = (Quizzes*0.2) + (Midsem*0.3) + (Final*0.5)

Then, it converts that number into a Letter Grade (like A or B) for the student to see.

---

5. Extras & Bonus Features

I added a few extra features to make the project better and more usable.

5.1 Change Password

I didn't want users to be stuck with the default password ("pass"), so I added a "Change Password" button.

- **It asks for the Old Password first.**
- **It uses BCrypt to check if the old password is correct. If it is, it hashes the new password and saves it. This prevents someone from changing a password if they don't know the current one.**

5.2 Export to CSV

I wrote a utility called CsvExporter.

- **This lets Instructors download their class list (names and grades) into a .csv file.**
- **This is useful if they want to open the data in Excel.**

5.3 Modern Look (FlatLaf)

The default Java look is a bit old-fashioned. I added a library called FlatLaf to make the buttons and windows look clean and modern, like a native Windows or Mac app.

---

6. Conclusion

This University ERP project meets all the requirements for the assignment. It shows that I understand Object-Oriented Programming (OOP) and how to connect Java to a MySQL database.

Using two databases makes the system more secure, and the "Maintenance Mode" shows how to manage the global state of an application. The grading logic was also a good exercise in handling data processing. The system is stable and ready to use.

---

7. References

- **Java Docs: Oracle Java SE 11 Documentation.**
- **Password Security: jbcrypt library documentation.**
- **UI Theme: FlatLaf for Java Swing.**
- **Class Notes: CSE 201 Lecture slides on JDBC.**
- **My Code:**
    - **src/main/java/edu/univ/erp/service/StudentService.java**
    - **src/main/java/edu/univ/erp/service/AdminService.java**

- src/main/java/edu/univ/erp/auth/AuthService.java

---

8. Sample Data for Testing

To test that everything works (like Enrollment and Grading), I added some sample data to the database. This includes 1 Admin, 1 Instructor, and 2 Students.

8.1 SQL Script

I used these SQL commands to fill the tables.

(Note: The passwords below are hashed versions of "pass")

SQL

```
-- 1. Create Users (Authentication)
INSERT INTO auth_db.users_auth (user_id, username, password_hash, role) VALUES
(1, 'admin1', '$2a$12$D7.vXy...', 'ADMIN'),
(2, 'inst1', '$2a$12$D7.vXy...', 'INSTRUCTOR'),
(3, 'stu1', '$2a$12$D7.vXy...', 'STUDENT'),
(4, 'achintya', '$2a$12$D7.vXy...', 'STUDENT');

-- 2. Create Profiles (ERP Data)
INSERT INTO erp_db.instructors (user_id, department, title) VALUES
(2, 'Computer Science', 'Assistant Professor');

INSERT INTO erp_db.students (user_id, roll_no, program, year) VALUES
(3, '2023001', 'B.Tech CSE', 2),
(4, '2023002', 'B.Tech ECE', 2);

-- 3. Create Course Catalog
INSERT INTO erp_db.courses (course_id, code, title, credits) VALUES
(1, 'CSE201', 'Advanced Programming', 4),
(2, 'MTH100', 'Linear Algebra', 4);

-- 4. Schedule Sections
-- Instructor 'inst1' teaches 'CSE201'
```

```
INSERT INTO erp_db.sections (section_id, course_id, instructor_id, day_time, room, capacity)
VALUES (101, 1, 2, 'Mon/Wed 10:00 AM', 'LHC-102', 60);

-- 5. Enrollments
-- Both students are enrolled in Section 101
INSERT INTO erp_db.enrollments (student_id, section_id) VALUES
(3, 101),
(4, 101);
```

8.2 Testing Results

After running this script:

- **Admin: admin1 can log in and see the 4 users.**
- **Instructor: inst1 sees the "Advanced Programming" class and a list of 2 students.**
- **Student: stu1 sees that they are already registered for "CSE201".**